

Документация сервиса подготовки релизов AIS

Дата составления: 03.08.2026

1. Назначение сервиса

Сервис расположен в каталоге `C:\AIS\AI\Prod\` и предназначен для автоматизации подготовки релизов корпоративной системы AIS (страховая учётная система на PowerBuilder 12.5 и Sybase ASE 15.5) с помощью больших языковых моделей (LLM) через инструмент OpenCode.

Основные задачи сервиса:

- выгрузка и сравнение объектов PowerBuilder (Current против Main) и SQL (dev_golden против galaxy);
- анализ и доработка объектов PB и SQL по задачам Jira (SYBASE-, SUPRT-);
- работа с системой контроля версий VSS (проверка статуса, извлечение, снятие резервирования);
- ведение базы знаний о проекте (объекты, бизнес-правила, связи, опыт решения задач);
- управление объёмными задачами с планами реализации и контрольными точками;
- автоматизированное тестирование диалоговых окон.

2. Компоненты сервиса (диалоговые окна)

2.1. МОРДА — главное окно подготовки релизов (СТАНДАРТ1)

- Файл: `bin\Prod-GUI.ps1` (лаунчеры: `.vbs` , `.bat`)
- Технология: PowerShell 5.1 + WPF
- Назначение: выполнение операций подготовки релиза — настройки, запуск тестов, выгрузка сервиса, сбор объектов для вывода в ПРОД, выгрузка и сравнение РВ и SQL, операции VSS и Jira.
- Журнал: `temp\last_output.log` (заголовок «=== AIS Output Log ===»), анализ командой ОБР/OUT.

2.2. МОРДА2 — новое поколение главного окна (СТАНДАРТ2)

- Файл: `bin\Prod-GUI_STANDART2.ps1` (лаунчеры: `.vbs` , `.bat`)
- Технология: PowerShell 5.1 + WPF, оформление СТАНДАРТ2 (тёмная шапка, панель прогресс-баров, панель кнопок).
- Ключевая особенность: асинхронное выполнение операций — каждая операция выполняется в фоновом Runspace (функция `Start-OpAsync`), интерфейс не блокируется; таймер `DispatcherTimer` (300 мс) опрашивает завершение и читает реальный прогресс операции из Runspace (переменные `PhaseProgress` и `StepProgress`), при отсутствии прогресса включается «пульс» до 90 процентов.
- Журнал: `%TEMP%\tech_journal_*.log` (первая строка «MORDA started (ST2)»).
- Автотест: `temp\AutoTest-Morda2-Real.ps1` — управление окном через UI Automation, проверка прогресс-баров, полный прогон девяти операций.

2.3. ObjectInfo — справочник объектов PB и SQL (СТАНДАРТ2)

- Исходник: `scripts\Show-ObjectInfo.cs` (C#, WPF, .NET Framework 4.0), скомпилированный файл: `bin\Show-ObjectInfo.exe`.
- Данные: `temp\pb_full.json` (500 объектов PB) и `temp\sql_full.json` (500 объектов SQL), выгружаются скриптом `temp\export_full.py` из базы `scripts\ais_objects_mcp\objects.db`.
- Возможности: две вкладки (PB и SQL) с таблицами объектов, фильтром и многоуровневой сортировкой; исходный код с поиском контекста; списки «Содержит», «Содержит SQL», «Содержится в»; свойства; построчное сравнение версий Main и Current (Merge) с кнопкой запуска TortoiseMerge; копирование данных с заголовками; контекстное меню коллекций «Открыть в окне».
- Автотест: `scripts\AutoTest-Pipe.ps1` — файловый канал команд `%TEMP%\objinfo_cmd.txt` / `objinfo_resp.txt` (команды ping, sel_pb, sel_sql, st_pb, st_sql, close).

2.4. TaskPlan — диспетчер объёмных задач

- Диалоговое окно: `scripts\Show-TaskPlanGUI.ps1` (команда запуска TaskPI).
- Трекер: `scripts\TaskPlan-Tracker.ps1` — ведение файла ПЛАН_РЕАЛИЗАЦИИ.txt с чек-боксами этапов и зеркала `plan_state.json`.
- Менеджер: `scripts\TaskPlan-Manager.ps1` — реестр множества задач (`task_index.json`): создание, переключение, приостановка, возобновление, правка промпта, удаление.
- Анализ ошибок: `scripts\Out-TaskPlan.ps1` (команда OUTPL) — распознавание восьми типов ошибок с автоисправлением; цикл автоисправления `Out-TaskPlan-FixCycle.ps1`.
- Автотесты: `scripts\TaskPlan-Tests.ps1` (15 тестов), `scripts\Out-TaskPlan-Tests.ps1` (10 тестов).

2.5. ПРОДОБР — окно продолжения обработки

- Файл: `scripts\Show-Prod0br-GUI.ps1` (команда ПРОДОБР).
- Назначение: продолжение работы по утверждённым планам — поиск задач со статусом «в работе» или «не начато» в `rules\pending-tasks.md` и папках задач, отображение прогресса по контрольным точкам, автопереход к следующей.

2.6. TimeFIX — календарь с интеграцией EWS

- Файл: `bin\Show-TimeFIX.exe` (C#, WPF, без PowerShell).
- Назначение: недельный календарь (Пн-Пт, 8:00-19:00) с цветными блоками мероприятий, загрузка встреч через EWS (Exchange Web Services), автопроверка «Дейли AIS», подтверждение мероприятий.
- Ранее использовал Outlook OLE/COM и PowerShell — заменён на C# + EWS для защиты от антивируса.

2.7. Setup-Secrets — сервис управления секретными данными

- Файл: `bin\Setup-Secrets.exe` (C#, WPF, СТАНДАРТ2).
- Назначение: ввод, шифрование (DPAPI) и сохранение паролей, токенов, ключей API при развёртывании проекта на других компьютерах. Замена устаревших секретов на новые.
- Вкладки: Sybase, VSS, Jira, OpenCode API, EWS (Exchange).
- Хранилища: `config\local_secrets.json` (зашифровано, в `.gitignore`) и `~\.config\ews-mscp\credentials.env` (вне репозитория).
- Журналирование: `tech_journal_*.log`. Автотест: `--autotest commands.txt`.
- Подробнее: `docs\setup-secrets-guide.md`.

2.8. Вспомогательные скрипты

Скрипт	Назначение
<code>scripts\Add-Bom.ps1</code>	Добавление BOM (UTF-8) к файлам .ps1 — защита от кракозябр и антивируса
<code>scripts\Preflight-Antivirus.ps1</code>	Массовая проверка скриптов на запрещённые паттерны, автофикс BOM
<code>scripts\ConvertTo-BatLauncher.ps1</code>	Конвертация .ps1 в .bat-лаунчер с base64 (обход детекта антивирусом)
<code>scripts\Convert-Docs.ps1</code>	Конвертация документации docs*.md в .html и .pdf
<code>scripts\Out-LastOutput.ps1</code>	Анализ свежего вывода МОРДА и МОРДА2 (команда ОБР/OUT)
<code>scripts\Save-Snapshot.ps1</code>	Архивирование проекта (команда СОХР)
<code>scripts\Save-UserPrompt.ps1</code>	Сохранение промптов пользователя в архив
<code>scripts\Export-PB.ps1</code> , <code>SQL_exp_single.bat</code>	Выгрузка объектов PB и SQL

3. MCP-серверы

MCP (Model Context Protocol) — протокол подключения внешних инструментов к LLM. Все серверы объявлены в файле `opencode.jsonc`, раздел «mcp».

3.1. ais-catalog — база знаний о проекте AIS

- Расположение: `scripts\ais_catalog_mcp\server.py` (Python, виртуальное окружение .venv).
- Назначение: главное хранилище знаний о корпоративном проекте — анализы задач Jira, бизнес-правила, метаданные объектов PB и SQL, связи между объектами, сленг аналитиков.

- Основные инструменты: сохранение и поиск знаний (save_knowledge, search_knowledge), бизнес-правила (add_business_rule, get_business_rules), метаданные и зависимости объектов (get_object_metadata, find_dependencies), поиск по кодовой базе (search_codebase), теги проектов и проверка кросс-проектного влияния (tag_object, check_cross_project_impact).
- Хранилище: SQLite с полнотекстовым поиском FTS5. Важно: FTS5 не знает русской морфологии, поэтому при сохранении обязательны явные теги (имена объектов, номера задач).

3.2. ais-objects — каталог объектов PB и SQL

- Расположение: `scripts\ais_objects_mcp\server.py` , база `objects.db` .
- Назначение: реестр всех объектов проекта (8464 объекта): PB-объекты из PB_Main (производственная версия) и PB_Current (песочница), SQL-объекты из BD\galaxy и BD\dev_golden.
- Основные инструменты: импорт объектов из файловой системы (import_objects), карточка объекта (get_object), список и поиск (list_objects, search_objects), сравнение версий Main и Current (compare_objects), граф зависимостей (find_dependencies, add_dependency), анализы и свойства объектов (save_analysis, get_analyses, set_property).

3.3. opencode-mcp — база знаний об инструменте OpenCode

- Расположение: `scripts\opencode_mcp\server.py` .
- Назначение: хранение опыта работы с самим инструментом OpenCode — настройка провайдеров LLM, шаблоны MCP-серверов, ошибки конфигурации, решения по WPF и PowerShell, история багов.
- Основные инструменты: save_knowledge, search_knowledge, get_knowledge, list_recent, delete_knowledge, knowledge_stats.

- Правило выбора хранилища: знания о проекте AIS — в ais-catalog, знания об OpenCode как инструменте — в opencode-mcp.

3.4. rules-mcp — правила проекта (только чтение)

- Расположение: `C:\AIS\AI\rules-mcp\run.py`.
- Назначение: быстрый доступ к статическим правилам проекта без чтения файлов — расшифровка сокращений (`get_abbreviation`), требования к кодировкам (`get_encoding_rule`), тексты правил по темам (`get_rule`, `get_full_rule`), предполётная проверка перед записью файла (`preflight_check`), поиск по истории багов (`search_bug`).
- Источники: `rules\abbreviations.md`, `rules\changelog.md`, файлы правил `.opencode\*.mdc`. В этот сервер ничего не сохраняется — он читает файлы напрямую.

3.5. sybase-docs — документация Sybase ASE 15.5 и PowerBuilder 12.5 (только чтение)

- Расположение: `C:\AIS\AI\sybase-docs-mcp` (Python, виртуальное окружение `venv`).
- Назначение: поиск по проиндексированной официальной документации Sybase — синтаксис, сообщения об ошибках, справочники.
- Основные инструменты: поиск по документации (`search_docs`), получение раздела целиком (`get_section`), чтение соседних фрагментов (`read_context`), список документов и продуктов (`list_docs_tool`, `list_products_tool`), проверка кода на соответствие документированным паттернам (`validate_code`).
- Пример применения: подтверждение причины ошибки 107 (несовпадение квалификаторов колонок и таблиц) по официальному тексту документации.

3.6. atlassian — Jira и Confluence (только чтение)

- Запуск: `uvx.exe mcp-atlassian` (готовый пакет).
- Назначение: чтение задач Jira (`https://mytask.renins.com`) и страниц Confluence (`https://wiki.renins.com`).
- Включённые инструменты: `jira_search`, `jira_get_issue`, `confluence_search`, `confluence_get_page`. Режим строго «только чтение» (`READ_ONLY_MODE = true`) — изменение задач и страниц запрещено.

4. Плагины OpenCode

Плагины объявлены в `opencode.jsonc`, раздел «plugin». Локальные плагины лежат в `.opencode\plugins\` и `.opencode\plugin\`.

4.1. @betterspec/opencode (внешний пакет)

Инструменты подготовки спецификаций: помогает формализовать требования до начала разработки.

4.2. @relf108/opencode-watchdog (внешний пакет)

Сторожевой контроль сессии разработки:

- блокирует правки кода до записи согласованного объёма работ в `.opencode\session\scope.txt` (каждая сессия начинается в режиме «только чтение»);
- ограничивает правки согласованным списком путей;

- требует базовый прогон тестов в начале сессии (test-baseline) и итоговый в конце (test-latest), сравнивает и сообщает о регрессиях;
- блокирует git commit при отсутствии файлов базовых прогонов;
- классифицирует команды оболочки и внешние адреса как безопасные или рискованные (агент watchdog-auditor);
- предупреждает о большом количестве правок (возможное заикливание).

4.3. save-prompts.js (локальный)

Сохранение всех пользовательских промптов в архив `archives\prompts\prompts_archive.log` через инструмент `save_prompt`. Защита от потери формулировок задач при смене LLM или закрытии сессии.

4.4. russian-compaction.js (локальный)

Русский язык при сжатии сессии: дополняет стандартный запрос сжатия контекста требованием формировать сводку по-русски (иначе сводки получаются на английском и нарушают языковое правило проекта).

4.5. claude-code.js (локальный)

Интеграция Claude Code как инструмента OpenCode: добавляет инструменты `claude_pb` (анализ и рефакторинг объектов PowerBuilder) и `claude_sql` (анализ хранимых процедур Sybase). Вызывает claude CLI в неинтерактивном режиме.

4.6. loop-guard.js (локальный)

Обнаружение и предотвращение заикливания: отслеживает повторяющиеся паттерны в ответах ассистента и при обнаружении цикла возвращает системное предупреждение.

4.7. task-progress.js (локальный)

Прогресс задач по промпту: при получении промпта показывает диалоговое окно Show-TaskProgressDialog через триггер-файл `temp\show_task_dialog.trigger`.

5. Провайдеры LLM

Настроены в `opencode.jsonc`, раздел «provider».

Провайдер	Статус	Модели	Примечание
routerai	основной	Claude Opus 4.8, Claude Sonnet 4.6, Claude Haiku 4.5, GPT-4o, DeepSeek V4 Pro, Gemini 2.5 Pro	Российский шлюз <code>https://routerai.ru/api/v1</code> , оплата по токенам
ollama	активен	Qwen 2.5 Coder 32B, DeepSeek Coder V2 16B	Локальные бесплатные модели, <code>http://localhost:11434</code>
yandex	отключён	YandexGPT	Проверка доступа
anthropic	отключён	Claude 3.7 Sonnet	Прямой доступ заблокирован сетью

Основная модель: Claude Opus 4.8, вспомогательная (small_model): Claude Haiku 4.5. У моделей заданы рейтинги пригодности для PowerBuilder (pb_rating) и SQL (sql_rating), стоимость и лимиты контекста — используются при выборе модели командой DLLM.

6. Правила и база знаний

- `AGENTS.md` — ядро правил: кодировки, команды, антивирус, стратегия решения объёмных задач недорогими LLM.
- `.opencode\*.mdc` — тематические правила: русский язык, предполётные проверки, РВ-объекты, SQL (включая правило об алиасах при dbo-квалификации — защита от ошибки 107), VSS, Jira, СТАНДАРТ2, глоссарий.
- `rules\abbreviations.md` — сокращения проекта (ДО, ОП, ПВКЛ, ПБ и другие).
- `rules\changelog.md` — история багов и исправлений (Bug-013 и далее).
- `rules\pending-tasks.md` — отложенные задачи с точками возобновления.

7. Структура каталогов

Каталог	Содержимое
<code>bin\</code>	Диалоговые окна (МОПДА, МОПДА2, ObjectInfo.exe, Show-TimeFIX.exe, Setup-Secrets.exe), лаунчеры
<code>scripts\</code>	Рабочие скрипты, MCP-серверы, C# проекты (Show-TimeFIX-CS, SetupSecrets, CountToken, PreCommitHook, ExportFull)
<code>config\</code>	<code>config.json</code> — пути проекта; <code>.local_secrets.json</code> — секреты (в <code>.gitignore</code>); <code>.local_secrets.example.json</code> — шаблон
<code>docs\</code>	Документация (md, html, pdf)
<code>rules\</code>	Правила, сокращения, история багов, отложенные задачи
<code>tasks\</code>	Папки задач (Git_, Test_, Ready_, Describe)
<code>temp\</code>	Временные файлы, журналы, данные ObjectInfo

archives\	Архивы проекта и промптов
BD\	Выгрузки SQL: galaxy (Main) и dev_golden (Current)
PB_Main\ , PB_Current\	Выгрузки исходников PowerBuilder
.opencode\	Правила, плагины, команды, сессия